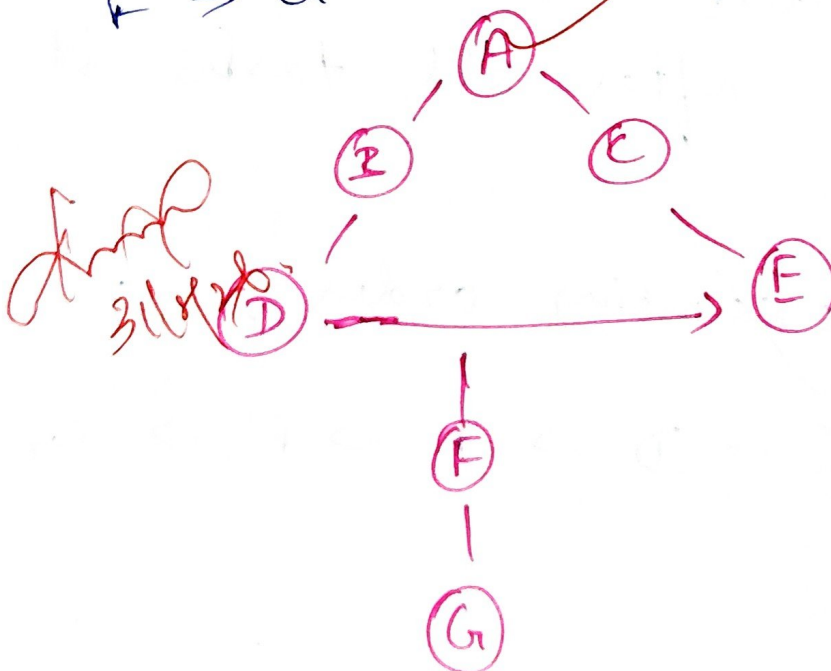


① Modeling the Problem as a Directed graph

- * Vertex $\rightarrow$ Represents a task
- * Directed edge $\rightarrow$ Represents a dependency between two tasks.
- * If there is an edge $A \rightarrow B$, task A must be completed before B.

Dependencies:

 $A \rightarrow B$ $A \rightarrow C$ $B \rightarrow D$ $B \rightarrow E$ $C \rightarrow E$ $D \rightarrow E$ $E \rightarrow F$ $F \rightarrow G$ $A \rightarrow$ Requirement Analysis $B \rightarrow$ System Design $C \rightarrow$ Database design $D \rightarrow$ UI design $E \rightarrow$ Implementation $F \rightarrow$ Testing $G \rightarrow$ Deployment

(b) Ensuring no Cyclic Dependencies

* $A \rightarrow B \rightarrow C \rightarrow A$. This is a cycle.

Such a dependency is impossible because A requires C , C requires B , B requires A .

None of these tasks can begin.

* If a topological ordering exists, the project has no cyclic dependencies.

(c) Generating a valid execution order

A topological sort produces an ordering of tasks such that every task appears after all tasks it depends on.

* One valid execution order is :

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

Another one:

* $A \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

Both are correct because B and C depend only on A and therefore can be performed in either order

Complexity

V = no. of tasks

E = no. of dependencies

Time Complexity:

$$O(V + E)$$

② DFS on a graph using an adjacency list.

③ Adjacency List representation

Eg:

A: B, C

B: D, E

C: E

D: E

E: F

F: G

G: -

A $\rightarrow$ B

A $\rightarrow$ C

B $\rightarrow$ D

C $\rightarrow$ E

D $\rightarrow$ F

E $\rightarrow$ F

F $\rightarrow$ G

An adjacency list is memory efficient particularly for sparse graph

⑥ DFS Pseudocode.

DFS (lgraph, vertex):

mark vertex as visited

print vertex

for each neighbor of vertex:

if neighbor is not visited:

DFS (lgraph, neighbor)

DFS (graph):

for each vertex V in lgraph:

if V is not visited:

DFS (graph, V)

DFS for Cycle Detection

DFS can also detect cycles by maintaining three states for every vertex:

State	Meaning
0	Unvisited
1	Currently being explored
2	Completely explored.

$A \rightarrow B \rightarrow C \rightarrow A$

When C finds A, A is already in the current DFS path.

$\therefore C \rightarrow A$ creates a cycle.

Pseudocode.

DFS - CYCLE (graph):

for every vertex v :

state $[v] = 0$

for every vertex v :

if state $[v] == 0$:

if visit (v) == True:

return TRUE

return FALSE

VISIT(V):

state[V] = 1

for each neighbour U of V:

if state[U] == 1:

return TRUE

if state[U] == 0:

if visit(U) == True:

return TRUE

state[V] = 2

return FALSE

(d) Relationship between DFS, cycle detection and task scheduling.

Project Tasks

↓
Directed graph

↓
DFS cycle detection

↓
Cycle No Cycle

↓
Error

↓
DAG

↓
Topological Sort

↓
Valid Task order

Thus the project manager can execute the tasks in this order while respecting all dependencies